Atlas RAG: System Reference

A fictional, text-based reference document for testing PDF ingestion, retrieval and grounded chat answers. These facts describe a test system, not your actual application.

1. Overview and ownership

Atlas RAG is a document question-answering service owned by the fictional Northstar Labs. Its project lead is Mira Patel. The internal project code is ATLAS-731. The backend uses FastAPI, PostgreSQL and a shared Chroma vector collection named atlas_documents.

Users register with a unique username and a password. Passwords are stored as hashes, never as plain text. A user ID is a UUID. Each document has its own UUID and a user_id foreign key identifying its owner. Every document read, search and delete request must check ownership before accessing content.

2. Upload and indexing

Atlas accepts TXT files and text-based PDF files up to 12 MB. Scanned PDFs without selectable text are not supported because Atlas does not perform OCR. Original uploads are stored on disk, while document metadata is stored in PostgreSQL.

During indexing, text is extracted, split into chunks of 800 characters with an overlap of 120 characters, embedded and added to Chroma. Each chunk includes document_id, user_id and chunk_index metadata. The embedding model is all-MiniLM-L6-v2. Indexing changes the document status from uploaded to indexing and then to indexed; errors change it to failed.

3. Retrieval isolation

Atlas does not rebuild the vector collection for each user. It reuses the shared collection. Before retrieval, the API validates that the requested document belongs to the authenticated user. Vector retrieval then filters by that document_id. Sharing a collection does not authorize sharing documents.

Atlas RAG: Answering and Cleanup

4. Retrieval and answer generation

For document questions, Atlas creates three alternate queries, performs semantic retrieval, combines results with Reciprocal Rank Fusion (RRF), and reranks the combined candidates with a cross-encoder. It passes the top four chunks to the answer generator. A smaller language model handles routing and grading; a larger language model generates the final answer.

A relevance grader checks whether retrieved chunks relate to the question. A grounding grader checks whether the generated answer is supported by those chunks. A usefulness grader checks whether the answer addresses the question. Graders return structured JSON fields containing lowercase yes or no.

If an answer fails the grounding check, the self-healing pipeline rewrites the query and repeats retrieval before generating another answer. It allows at most two retries after the initial attempt, for three generation attempts in total. If those attempts are exhausted, it returns: I could not produce a grounded answer from this document.

5. Deletion and sessions

Deleting a document removes its Chroma vectors, its chunk JSON file, its original uploaded file and its PostgreSQL document row. Deleting a user also removes that user's documents and their corresponding files and vectors. A database cascade deletes related database rows only; it does not delete disk files or Chroma vectors. Application cleanup handles those external resources.

Chat messages belong to a login session and are deleted when the user explicitly logs out. Closing the browser is not treated as a confirmed logout. Session expiration also triggers cleanup in Atlas.

6. Boundaries of this reference

This document does not specify a monthly subscription price, a production deployment region, a JWT signing secret, a database password or any real user credentials. An answer claiming specific values for those items is not grounded in this reference. No web lookup is needed to answer questions about the fictional Atlas system.